

# Proof notes: what each result actually says and why it's true

Hoang Nguyen

## Abstract

The theorems below mostly say things that sound like they should just be true — “merging two regions doesn’t create a duplicate id”, “a field write somewhere active can’t disturb a suspended region”, “once a region is closed, the only way in is through the front door”. Intuition gets you the headline for free. It does *not* get you the proof: in every case below, intuition is either silently assuming a fact that has to be derived from the model’s invariants first, or is flatly wrong until qualified in a way that isn’t obvious until you try to break it. This document’s job is to make that gap visible — what a proof actually had to rule out that intuition glosses over — for someone who trusts the model’s design (`writeup/main.tex`, `lean/gc/README.md`) but hasn’t looked at how any of these results got proved.

Format: each result gets *the claim, what your intuition already buys you, where the real work is*, and *how the proof closes that gap* — written so each one can be read and retold on its own. A reference catalog of the reusable lemmas and known pitfalls follows at the end, for looking things up once you already know which proof you’re asking about.

Scope: `Gc/Model/` and `Gc/Reachability/Reachable/`. `Gc/Reachability/Referencable/` is deliberately excluded — it is deprecated and superseded by `Reachable/` (see `CLAUDE.md`).

## Contents

|          |                                                                           |          |
|----------|---------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Preliminaries</b>                                                      | <b>2</b> |
| 1.1      | The model and its rules . . . . .                                         | 2        |
| 1.2      | The ten invariants of <i>ValidConfig</i> . . . . .                        | 2        |
| 1.3      | <i>RuntimeConfig.start</i> satisfies <i>ValidConfig</i> . . . . .         | 3        |
| 1.4      | The preservation proof pattern . . . . .                                  | 3        |
| <b>2</b> | <b>How the proofs actually work</b>                                       | <b>3</b> |
| 2.1      | <code>merge</code> keeps the configuration valid . . . . .                | 3        |
| 2.2      | Reachability, recapped . . . . .                                          | 4        |
| 2.3      | “You can’t reach backward in time” . . . . .                              | 5        |
| 2.4      | What “Closed” really buys you . . . . .                                   | 6        |
| 2.5      | A field write in the active frame . . . . .                               | 6        |
| 2.6      | Deleting a region reference . . . . .                                     | 7        |
| <b>3</b> | <b>Reference: everything else</b>                                         | <b>8</b> |
| 3.1      | The other operations . . . . .                                            | 8        |
| 3.2      | <i>FRALF</i> preservation: the three proof shapes, in one place . . . . . | 9        |
| 3.3      | Reusable lemma catalog . . . . .                                          | 9        |
| 3.4      | Pitfalls . . . . .                                                        | 10       |
| 3.5      | What’s deliberately not done here . . . . .                               | 11       |

# 1 Preliminaries

## 1.1 The model and its rules

The configuration  $\langle S, H \rangle$  (stack of frames, heap of regions) and the nine mutation operations’ operational-semantics rules are given in full in `writeup/main.tex`, in its “Model” and “Operational Semantics” sections.<sup>1</sup> We reuse that notation throughout:  $\text{Rld}(rid)/\text{Old}(oid)$  for references,  $a.\text{loc}(\langle S, H \rangle)$  for where a reference resolves ( $\text{Stk}(fid)$  or  $\text{Rgn}(rid)$  or undefined),  $a.\text{objAt}(\langle S, H \rangle)$  for the object an Old reference currently names.

## 1.2 The ten invariants of *ValidConfig*

`main.tex`’s “Invariants” section defines all ten ( $L1$ ,  $L2$ ,  $H1$ ,  $H2$ ,  $H3$ ,  $S1$ ,  $S2$ ,  $S3$ ,  $HS1$ ,  $HS2$ ) and bundles them into *ValidConfig*. We do not restate the formulas here; the table below is only a one-line reminder of what each one *forbids*, since that is the fact every proof below actually reaches for.

| Invariant | What it rules out                                                                |
|-----------|----------------------------------------------------------------------------------|
| $L1$      | the same object id allocated twice (stack or heap, anywhere)                     |
| $L2$      | a frame executing inside a region that is actually <i>Closed</i>                 |
| $H1$      | a region whose bridge object isn’t actually a member of its own <i>objMap</i>    |
| $H2$      | two live $\text{Rld}(rid)$ references to the same region                         |
| $H3$      | a region’s contents holding a reference that resolves <i>outside</i> that region |
| $S1$      | two stack frames both claiming the same region                                   |
| $S2$      | a frame referencing a stack object owned by a <i>later</i> frame                 |
| $S3$      | a frame referencing a region owned only by a <i>later</i> frame                  |
| $HS1$     | a dangling Old (points at an id never/no-longer allocated)                       |
| $HS2$     | a dangling Rld (points at a region no longer in the heap)                        |

**Remark 1.1.**  $HS1/HS2$  are not derivable from the rest:  $H2$ ,  $H3$ ,  $S2$ ,  $S3$  all presuppose a reference that *already* resolves somewhere; none of them constrains a reference that resolves to nothing. Without  $HS1/HS2$ , a stale reference could sit inert and then coincidentally collide with a freshly allocated id later, and nothing above would have caught the staleness in between. (`main.tex` makes the same point; it surfaced while proving  $S2/S3$  preservation for the three `make*` operations, whose `freshObjectId/freshRegionId` only look at ids currently used as *keys*.)

Four corollaries proved directly from the invariants (`Validity.lean`) are cited constantly by everything downstream, in both layers:

- `oid_in_stack_implies_not_in_heap / oid_in_heap_implies_not_in_stack` — from  $L1$ : an id found by one search can’t also be found by the other.
- `oid_loc_rgn_iff_in_heap, oid_loc_stk_iff_in_stack, rid_loc_rgn_iff_in_heap` — rewrite the opaque computation  $\text{Old}(oid).\text{loc}(\langle S, H \rangle) = \text{Rgn}(rid)$  into the usable  $\exists \text{region}. H(rid) = \text{region} \wedge oid \in \text{region.objMap}$ . Every later file that needs to reason about *where* an id lives goes through one of these three, never through the raw definition of  $\text{loc}(\cdot)$ .

<sup>1</sup>This document compiles standalone, so cross-file `\refs` to `main.tex`’s labels are not used; section names are given in prose instead.

### 1.3 *RuntimeConfig.start* satisfies *ValidConfig*

One root region (Open, bridge object 0) and one root frame executing in it. Each of the ten invariants reduces, by unfolding, to a fact about a one- or two-element list, closed by `decide/simp/List.nodup_singleton`. No interesting content — this exists purely to anchor the induction below.

### 1.4 The preservation proof pattern

Every `Gc/Model/Mutation/<Op>.lean` defines the operation as an *Option RuntimeConfig*-valued `do`-block (each `bind` is a precondition that can fail — exactly the premises of the corresponding inference rule in `main.tex`), paired with an `<op>_cases` lemma that unpacks a successful run into an explicit disjunction of fully-instantiated hypotheses, one disjunct per inference-rule variant. This is mechanical but load-bearing: it is, in effect, the operation’s real specification — a mistake here (a missing disjunct, a wrong side condition) would not surface as a Lean error in its own short proof (which just replays the `do`-block’s control flow); it would surface only as an unprovable, or vacuously provable, downstream theorem. Every proof from here on, in both layers, destructs a hypothesis  $op\ cfg = some\ cfg'$  via `<op>_cases` exactly once, then argues with concrete named values for the rest.

`Preservation/<Op>.lean` then proves ten lemmas `<op>_L1, ..., <op>_HS2` and combines them into `<op>_valid : ValidConfig(cfg) → op cfg = some cfg' → ValidConfig(cfg')`. `Preservation.lean` aggregates all nine into `AllPreserve(ValidConfig)` by case-splitting on the reified *Stmt* type (`Mutation/Stmt.lean`) — the statement “every one of the nine operations preserves *ValidConfig*” proved once, generically, rather than left as nine theorems a caller has to know to look up individually.

## 2 How the proofs actually work

Five walkthroughs, chosen to cover both layers and every genuinely distinct proof technique in the codebase. §2.1 needs only §1.2 above. §2.3–2.6 are about `Gc/Reachability/Reachable/`, so §2.2 first recaps just enough of that layer’s definitions to make them self-contained (all stated in full in `main.tex`).

### 2.1 merge keeps the configuration valid

**The claim.** After `merge x` folds a Closed region `rid'` into the current Open region `rid` (combining their *objMaps*, deleting `rid'` from the heap, rebinding `x` to `rid`’s old bridge object), the result is still a *ValidConfig* — in particular still has no duplicate object ids (*L1*), and every region’s contents still resolve back into that same region (*H3*).

**What your intuition already buys you.** “We’re just pouring one region’s objects into another; nothing is created or destroyed, so of course ids stay unique, and anything that already pointed home still points home.” This is directionally right, and it’s the whole content of the theorem as far as a reader is concerned — but “nothing is created or destroyed” is a claim about the operation’s *semantics*, and the proof has to earn it from the operation’s precondition, not assume it from the English description of `merge`.

**Where the real work is.**

- **Disjointness isn’t given, it has to be derived.** Nothing in `merge`’s `do`-notation says the two regions’ object-id sets are disjoint. If they weren’t, the union could either silently

identify two previously-different objects (breaking *L1* the *other* way), or — since *AList.union* is implemented via *kunion/kerase*, which erase duplicate keys from one side — silently drop an object nobody asked to delete. Disjointness has to be derived from the *only* fact the precondition actually gives you: one region is Open, the other Closed, and Open/Closed are mutually exclusive statuses, so *L1* (no id in two heap entries) rules out any shared id between them.

- ***H3* isn’t a re-labelling, it’s a fresh search.** An object that used to live in the now-*deleted* region *rid*’ needs to be shown to resolve, post-merge, into *rid*. But *.loc(·)* doesn’t remember history — it performs a brand-new search over whatever heap exists *right now*. So the proof has to reconstruct, from nothing, that searching the post-merge heap for this object’s id lands on *rid* and nowhere else — which again needs disjointness, now to rule out a second heap entry also matching.

### How the proof closes the gap.

1. *merge\_corollary\_rid\_ne\_regionId*: *rid* and *rid*’ are provably distinct heap entries, from *Open*  $\neq$  *Closed*.
2. *merge\_corollary\_region\_unique* (an *L1* corollary): no object id can appear in two different heap entries’ object maps — so the two regions-to-be-merged share no id.
3. *merge\_corollary\_union\_append*: *on disjoint keys*,  $s_1 \cup s_2$ ’s *.entries* is literally  $s_1.entries ++ s_2.entries$  — this is what licenses treating the merged region’s contents as “*region*’s own entries, then *region*’s, nothing else,” rather than trusting *AList.union*’s general (lossy) behaviour.
4. *merge\_corollary\_heap\_entries\_perm*: the whole post-merge heap’s entry list is a *permutation* of “the merged pair, plus everything untouched” — built by pulling *rid* and *rid*’ out of the original heap one at a time (two nested *List.exists\_of\_kerase* extractions).
5. *L1* then reduces to permutation-invariance: *List.count/Nodup* don’t care about reordering, so “the merged heap’s object-id list is a permutation of the original” is enough.
6. *H3*: *merge\_corollary\_loc\_of\_mem* shows an affected object still resolves into *rid* post-merge by showing (a) the stack side of *.loc(·)*’s search is untouched (*merge* only rewrites the current frame’s *varMap*, never its *objMap*), and (b) on the heap side, the uniqueness from steps 2/4 pins down that *rid*’s merged entry is the *only* heap entry whose *objMap* could contain this id.

**The one-line version.** The hard part of *merge* was never the union itself — it was proving, from the operation’s own precondition, that the two regions being combined could never have shared an id in the first place, and then re-deriving from scratch where every affected object resolves under the brand-new heap, because *.loc(·)* has no memory of where anything used to be.

## 2.2 Reachability, recapped

$a \rightarrow b$  (*ReachableStep*) is one hop of a reference chain; the only content beyond following an object’s own stored references is:  $\text{Rld}(rid) \rightarrow b$  is possible only when  $H(rid)$  is *Closed*, and forces  $b = \text{Old}(H(rid).bridgeObjectId)$  — a *Closed* region’s *Rld* has exactly one outgoing step, into its own bridge object; an *Open* region’s *Rld* has none at all (*L2*: if it could step it would have to be *Closed*, contradicting an on-stack frame owning it). This one asymmetry — a chain may step *into* a suspended region through this single portal, but never out — is the entire reason this layer exists, and it is what makes §2.4 non-trivial.

$RegionReachable(rid, \cdot) / FrameReachable(fid, \cdot)$  are the inductive closures of  $\rightarrow$  from a region’s bridge object / a frame’s roots (a variable’s value, or that frame’s own region’s bridge object —  $FrameRoot$ );  $StackReachable$  is “ $FrameReachable$  from *some* on-stack frame.” All of `main.tex`’s  $\rightarrow^*$ -reformulation theorems restate these as “exists a root, and a `Relation.ReflTransGen`( $\rightarrow$ ) chain from it” — and it is this restated form, not the raw inductive definition, that every proof below actually inducts on. `ReflTransGen` comes with two mathlib induction principles: ordinary constructor induction (fixes the chain’s *first* element, grows the *last* one step at a time — right for “trace forward from a known root”) and `head_induction_on` (fixes the *last* element, peels off the *first* step — right for “chase an escaping chain hop by hop from a known start”). Picking the wrong one doesn’t produce a Lean error; it produces a stuck goal whose motive silently fails to generalize the variable the argument needs.

### 2.3 “You can’t reach backward in time”

**The claim** (`region_container_confined`, `stack_container_confined`). If you can trace a reference chain, from some frame at index  $fid$ , back to an object sitting inside a particular (Open) region — or a particular stack frame — then whichever frame actually *owns* that container has index no later than  $fid$ .

**What your intuition already buys you.** “You can’t reach an earlier/more-nested frame’s stuff from a frame that comes later, unless something already linked the two” sounds like it should be close to definitionally true from the stack discipline — after all,  $S2/S3$  already say exactly this.

**Where the real work is.**  $S2/S3$  are *one-hop* facts: they bound a single step of a chain. The claim above is about a chain of arbitrary, a priori unknown length, handed to the proof only as an opaque `ReflTransGen` witness. Getting from “true for one hop” to “true for the whole chain” is not free — it needs an induction that walks the chain, re-establishing the bound at every hop, and it needs a genuine case split for the one moment a chain might *cross* from inside the region onto the stack, which is a qualitatively different kind of hop from staying inside.

**How the proof closes the gap.**

1. Restate the  $FrameReachable$  witness as its  $ReflTransGen$  form (§2.2) — the custom inductive relation’s own recursion runs the wrong direction for this argument.
2. Generalize over the chain’s end (the reachable object) and induct *forward* from the fixed start. At each step, either (a) the chain is still inside the region — `predecessor_of_region_object` (an  $H3$ -driven fact: for an *Open* region, anything one hop away from an in-region object is itself in that same region or has escaped to the stack, never a bare `RId` or a *different* region) lets you recurse — or (b) the chain has just escaped to the stack, at which point the one-hop bound (`stack_step_region_index_le`, i.e.  $S3$  restated at the single-hop level) gives the numeric bound directly and the induction stops.
3. The two ways a chain can *start* (a variable slot, or a frame’s own bridge object) are each closed off by one more application of  $S3$  plus `merge_corollary_regionId_unique_index` (turning “same region” into “same index”).

**The one-line version.** This lemma is where “you can’t reach backward in time” stops being an informal slogan and becomes an actual proof — and the reason it needed real work is that the chain is abstract and unboundedly long, so a one-hop fact has to be bootstrapped into an arbitrary-length

one by induction, with a genuine case split at the exact moment a chain would cross from inside a region onto the stack.

## 2.4 What “Closed” really buys you

**The claim (*CR6*).** Once a region is Closed: if nothing outside it already holds its front door (Rld), *nothing* inside it is stack-reachable at all; and if something does hold the front door, stack-reachability of anything inside is *exactly* region-reachability from the bridge object.

**What your intuition already buys you.** “Closed means sealed; you get in through the front door or not at all” is more or less the entire informal meaning of “Closed”, so this can feel like it should be nearly a restatement of the definition.

**Where the real work is.** *ReachableStep*’s definition only tells you the *forward* direction: a Closed region’s Rld has exactly one successor. It says nothing about *predecessors* — i.e. nothing directly rules out some other, unforeseen way of already being inside. The actual content of “sealed” is a *negative* claim (there is no other way in), and proving a negative means enumerating and closing off every other conceivable entrance: another region’s object secretly holding a pointer in (would need *H3* to fail, since *H3* says a region’s own stored references always resolve back into that same region — so nothing external can hold a live pointer *into* it), and a stack frame that’s “still” somehow executing there (would need *L2* to fail — an on-stack frame can never own a Closed region in the first place).

**How the proof closes the gap.**

1. *predecessor\_of\_closed\_region\_object*: the Closed mirror of §2.3’s backward-confinement lemma, but now the “escaped to the stack” branch is *impossible* (via *L2*’s contrapositive, *closed\_region\_not\_owned*) rather than a base case, and the Rld branch — the predecessor is the region’s own portal — becomes *available* rather than something to rule out.
2. *RegionReachable\_of\_FrameReachable\_closed* / *FrameReachable\_rld\_of\_closed\_container*: a *backward* induction over an arbitrary chain, dispatching at every step on (1)’s two surviving cases — either still tracing inside the region (recurse), or the chain has just used the one portal hop (stop; the remaining tail of the chain, back to its root, is itself a witness that the region’s own Rld was already reachable).
3. *CR6* assembles these directly: stack-reachability of anything inside a Closed region factors uniquely through the portal.

**The one-line version.** The definition of Closed only tells you the one way in that’s *allowed*; *CR6* is the proof that it’s also the *only* way in — and that negative claim needed two separate invariants (*H3*, so no outside object can hold a pointer in; *L2*, so no live stack frame can still be the owner) to rule out the two entrances a proof-by-definition-alone wouldn’t have covered.

## 2.5 A field write in the active frame

**The claim (*FRALF* preserved by *fieldAsgn*).** If an object living in an earlier (possibly suspended) frame’s region was already visible from every later frame before a field write happening in the currently-active frame, it stays visible from that earlier frame afterward.

**What your intuition already buys you.** “A suspended region’s contents aren’t touched by a field write happening somewhere else entirely” is the natural first reaction, and it’s *almost* the whole truth — which is exactly what makes it dangerous: the natural reaction stops one level too early.

**Where the real work is — the one place intuition is flatly wrong if left unqualified.** The write *does* create one brand-new edge in the reference graph ( $obj.field := newValue$ ), and nothing about “the write happens somewhere active” rules out, *syntactically*, that this new edge connects two things that weren’t connected before, or shortcuts a longer existing detour. The claim isn’t true because the new edge is irrelevant to the suspended frame’s own chains by assumption — it’s true, and has to be *shown* true, because anything already visible from an earlier, strictly-suspended frame is (by §2.3) already confined to have a root no later than that suspended frame itself, which is strictly earlier than where the write happened — so the new edge, whatever it connects, is provably out of reach of any chain that could matter here.

**How the proof closes the gap.**

1. Let  $oid_C$  be the mutated object’s id. By §2.3’s lemma — now applied to  $oid_C$  itself rather than to the tracked object — anything that can reach  $oid_C$  at all must come from a frame index no earlier than the active frame’s own index,  $fid_{active}$ .
2. Any chain confined strictly below  $fid_{active}$  therefore never touches  $oid_C$ , so the step relation is completely untouched along it, and it transports across the mutation for free. (This covers every suspended frame’s chain automatically, since a suspended frame’s index is, by hypothesis, strictly below the active one.)
3. The one case needing genuine work: a chain rooted *exactly* at the active frame (which is free to have used the new edge). Walk this chain forward hop by hop (`Relation.ReflTransGen.head_induction_on`), asking at each hop “was this the newly-written one?” If yes, the chain has jumped onto the field write’s *new* value, which has its own, entirely independent root (`resolveV_frameRoot`) — and the argument restarts from there, landing back in the confined, already-handled case of step 2.

**The one-line version.** The field write’s new edge really could matter in general — the proof doesn’t get to assume it away, it has to *show* that only one specific frame’s own chain could ever route through it, and even that one case reduces back to an ordinary confined argument by tracing the new edge’s endpoint to its own separate root.

## 2.6 Deleting a region reference

**The claim (*FRALF* preserved by merge).** Same headline property as §2.5, now for the one operation that deletes an entire region,  $rid'$ , along with the single live reference ( $Rld(rid')$ ) that named it.

**What your intuition already buys you.** “merge doesn’t create any new objects, it just relabels which heap key some existing objects live under — so of course nothing about reachability from an earlier frame changes.”

**Where the real work is.** This undersells exactly one thing: a reference that used to exist —  $Rld(rid')$ , sitting in the merging frame’s own variable  $x$  — is *deleted outright* and replaced by

a reference to `region'.bridgeObjectId`. Any reachability argument that (even implicitly) ever routed through the old reference needs to be re-derived through the new one — and “nothing used it” can’t just be asserted, it has to be shown that nothing *could* have used it except as a chain’s very first step.

### How the proof closes the gap.

1. `merge_ridPrime_not_step_target`:  $\text{Rld}(rid')$  can never be the *target* of a step. If some reference stepped into it,  $H2$  (at most one live reference to any region) would already be violated, since  $x$ ’s own occurrence is already the one allowed one. A purely counting-based fact — no case analysis on chains needed.
2. Consequently  $\text{Rld}(rid')$  can only ever be a chain’s *root*, never mid-chain, so “did this chain use the deleted reference” reduces to one question about the chain’s *start*, not a hop-by-hop trace.
3. `merge_chain_transport_down`: any chain whose root avoids  $\text{Rld}(rid')$  transports unconditionally between the two configurations.
4. The one remaining case — a chain rooted *exactly* at  $\text{Rld}(rid')$  — is handled by `merge_ridPrime_reflTransGen_iff`: since  $\text{Rld}(rid')$ ’s only possible first hop is, by definition, its own bridge object, such a chain is *the same chain, one hop shorter*, rooted at `region'.bridgeObjectId` instead — exactly the value  $x$  now holds post-merge.

**The one-line version.** `merge` needs its own argument, distinct from an ordinary field write, because it *deletes* a reference rather than editing one — and the proof exploits an existing invariant ( $H2$ ’s one-reference-per-region bound) to show the deleted reference could never have been anything but a chain’s starting point, turning a potentially open-ended re-routing problem into a one-line substitution.

## 3 Reference: everything else

The rest of `Gc/Model`’s nine operations, the reusable lemma catalog, and known pitfalls — for looking things up once you already know which proof you’re asking about, rather than for reading start to finish.

### 3.1 The other operations

`varAsgn x yf` Two branches on whether  $x$  is the frame’s own bridge variable. If so, this is really a *bridge-object reassignment* — subtle because it changes  $H1$ ’s witness for that region without touching `objMap` at all; the proof shows the *new* bridge id is still a key of the (unchanged) `objMap`, which falls straight out of the rule’s own side condition ( $yfRef.loc(\cdot) = \text{Rgn}(rid)$  with  $rid = frame.regionId$  — you can only rebind the bridge to something already inside the same region). If not the bridge var, it’s a plain local write gated by  $resolveV(x, \cdot) = \epsilon$  (no existing binding), which exists purely to keep `resolveV` (outward frame search) unambiguous.

`fieldAsgn xf y` The `field-asgn-stack/field-asgn-region` split is the clearest illustration of why  $H3$  needs an explicit *status = Open* side condition: a region write is permitted only while the writer’s frame executes *in* that exact region *and* the new value resolves into the *same* region. Drop either equality and  $H3$ ’s preservation becomes false — nothing else in the rule would stop the region from acquiring an outward-pointing reference. (Its reachability-layer proof is §2.5 above.)



**swap**  $x\ yf$  Syntactically the busiest rule (two locations get new values simultaneously) but its invariant proofs are **fieldAsgn**’s field-write argument and **varAsgn**’s var-write argument run side by side — no new invariant-level content.

**makeObjStack**  $x$  / **makeObjRegion**  $x$  Allocate a fresh, empty object at  $\langle S, H \rangle.freshObjectId$ .  $L1$  preservation is exactly **RuntimeConfig.freshObjectId\_not\_mem** (**Helpers.lean**): the fresh id is, by construction, not already in  $cfg.objectIds$ .  $H2/HS1$  for the new id are vacuous (nothing points at it yet). **makeObjRegion** additionally needs  $status = Open$  — an object can only be born directly inside a region while someone is executing there.

**makeRegion**  $x$  Allocates a brand-new *Closed* region with its own fresh bridge object, and stores an **Rld** to it in the current frame.  $H2$  for the new region id is again vacuous by freshness (**freshRegionId\_not\_mem**); starting *Closed* is what keeps  $L2$  trivially true — nobody is executing inside the new region yet.

**merge**  $x$  See §2.1 (*ValidConfig* preservation) and §2.6 (the reachability-layer proof).

**enter**  $xf$  **bridgeVar** / **exit** Push/pop a frame, flipping the entered/exited region’s status.  $L2/S1$  are index bookkeeping; the freshly pushed frame starts with empty *objMap/varMap*, so  $S2/S3/HS1$  need say nothing new about it — its only content is the just-flipped bridge object, already covered by the region-side invariants. On the reachability side, both are “additive” operations (§3.2): the object graph’s content doesn’t change, only which frames/regions are active, so the uniform safety-transport tool (**safe\_reflTransGen\_transport**) applies directly rather than needing a per-hop induction.

### 3.2 *FRALF* preservation: the three proof shapes, in one place

§2.5 and §2.6 are two of the three shapes the nine per-operation *FRALF*-preservation proofs take; for completeness, all three:

1. **Additive / status-only** (**enter**, **exit**, **makeObjStack**, **makeObjRegion**, **makeRegion**, **varAsgn**) — no existing object’s content changes, so **safe\_reflTransGen\_transport** (§3.3) moves any chain across the mutation with no per-hop case analysis.
2. **Content-mutating** (**fieldAsgn**, **swap**) — §2.5’s escape argument, keyed on an owner-index bound.
3. **Reference-deleting** (**merge**) — §2.6’s “can only be a root” argument, keyed on an  $H2$  count bound.

The corresponding *StackReachable*-invariant proofs (**StackReachableInvariant**/) use *exactly* the same case analysis per operation, now proved as an  $\iff$  rather than a one-way implication: “was *oid* reachable before” and “is it reachable after” are genuinely symmetric questions once confined transport exists in both directions, but the two directions of the escape argument are *not* mirror images of each other and must be proved independently — the newly-written edge exists in  $cfg'$  but not  $cfg$ , so the escape case only genuinely arises in one of the two directions.

### 3.3 Reusable lemma catalog

**stackWithIndex\_frame\_transport\_down\_of\_shape\_eq** / **\_up\_of\_shape\_eq** If  $cfg, cfg'$  agree up to some projection  $\pi : Frame \rightarrow \alpha$  at every stack *position*, *stackWithIndex* membership transports between them, preserving *index* and the projected value.

**stackWithIndex\_objMap\_get\_eq\_of\_last\_varMap\_update** “Only the last frame’s *varMap* changed”  $\Rightarrow$  *objMap* is unaffected at every position. Used by every pure var-write operation.

`merge_corollary_regionId_unique_index` *S1* restated at the *FrameWithIndex* level: two frames sharing a *regionId* are the same frame. Used constantly to convert “same region” facts into “same index” facts.

`swap_corollary_stackWithIndex_index_inj / _find_eq` Two frames with the same *index* are the *same* element (*index* is assigned by position, not stored data). Arguably the single most-cited lemma in *Gc/Reachability/Reachable/* — needed whenever a proof juggles several differently-named frame variables that are provably-but-not-syntactically the same frame.

`predecessor_of_region_object / _of_closed_region_object` Backward confinement, one hop, Open/Closed — see §2.3/§2.4.

`SafeRef / predecessor_safe / safe_reflTransGen_transport` A reference is “safe” for region *rid* if it’s in *rid* or on the stack (never a bare *RId*). Safety propagates backward one hop; if the step relation agrees on every Old-sourced step between *cfg, cfg'*, any chain ending at a safe target transports wholesale. This is the tool behind the “additive” proof shape (§3.2).

`stack_container_confined / region_container_confined` The index-bound lemma of §2.3.

`RegionReachable_of_FrameReachable_closed / FrameReachable_rid_of_closed_container` The Closed-region confinement engine behind *CR6* (§2.4).

The “*H2* forbids a reference from being a step target” trick (§2.6) generalizes beyond *merge*: any future operation that *deletes* a uniquely-referenced region reference can reuse the same shape (count-based non-membership  $\Rightarrow$  chain-position restriction  $\Rightarrow$  transport reduces to a root-only case split).

### 3.4 Pitfalls

**Pitfall 3.1** (*AList.insert* reorders on an existing key). Inserting at an already-present key goes through *kinsert/kerase* and can reorder *.entries*, unlike inserting at a genuinely fresh key. Any proof needing *.entries*-level equality after an *insert* (not just *.lookup*-level) — *merge*’s region-combining argument (§2.1) — has to go through an explicit *List.Perm* argument (*exists\_of\_kerase + NodupKeys.eq\_of\_mk\_mem*), never assume “insert = append.”

**Pitfall 3.2** (*Status* has no *LawfulBEq*). *Status* derives *BEq/DecidableEq* but not *LawfulBEq*, so *beq\_iff\_eq* doesn’t fire on it. Every “Open xor Closed” contradiction in this codebase is closed by case-splitting *region.status* directly and *decide*, never by rewriting a *==* hypothesis.

**Pitfall 3.3** (the *<op>\_cases* lemma is the semantics). Because the operation is opaque *do*-notation, when in doubt about an operation’s actual precondition, read its *<op>\_cases* lemma’s statement, not the *do*-block — a mistake in the former is invisible until a downstream theorem quietly stops being provable.

**Pitfall 3.4** (Open-only tools misapplied to a Closed region). §2.3’s index-confinement lemma (region variant) takes *status = Open* as a hypothesis, because its proof leans on *predecessor\_of\_region\_object*, which is stated only for the Open case (an object inside a *Closed* region has no stack-escape branch to bound in the first place). For a Closed region the right tool is §2.4’s confinement lemmas instead. Reaching for the Open-only lemma on a Closed region fails to apply outright rather than silently giving a wrong bound — but it is an easy mismatch when adapting an existing per-operation proof to a new operation.

**Pitfall 3.5** ( $loc(\cdot)$  is treated as opaque). The raw definition of  $a.loc(\langle S, H \rangle)$  (nested reverse-stack-search / heap-search with a match forcing exactly one to succeed) is unfolded exactly once, in `Validity.lean`, to prove the three `_iff_in_*` restatement lemmas of §1.2. Every later file treats  $loc(\cdot)$  as an opaque function characterized by those iffs, never by unfolding its definition again.

**Pitfall 3.6** (picking the wrong `ReflTransGen` induction principle). Ordinary constructor induction fixes the chain’s start and grows its end; `head_induction_on` fixes the end and peels off the start. Using the one that doesn’t match what the argument actually needs to generalize doesn’t error — it produces a stuck goal several tactics later. See §2.2.

**Pitfall 3.7** (an  $\iff$  needs both halves proved independently). It is tempting to think the  $cfg' \rightarrow cfg$  direction of an escape argument is just the  $cfg \rightarrow cfg'$  direction “with the arguments flipped”. It is not: the newly-written edge exists in  $cfg'$  but not  $cfg$ , so the escape case genuinely only arises in one of the two directions and has to be proved on its own terms there. See §3.2.

### 3.5 What’s deliberately not done here

No analogue of a `Guarantees.lean`-style top-level GC-safety theorem sits on top of *StackReachableInvariant* yet. Everything in §2 is a *single-step* fact; composing it over an arbitrary-length trace — the literal `main.tex/report.pdf` claim, which talks about invariance for as long as a region stays suspended, not just across one operation — is not currently being pursued.